Week 4 Day 2: Indexed Analytics & Automation

CST4714 Database Administration

Agenda

- Reset the lab dataset and confirm the schema
- Benchmark one query before and after an index
- Learn window functions in three short passes
- Save the query as a view and share it safely
- Refresh summary data with a simple trigger
- Build an advisor-friendly function from the pieces

Resetting the Lab Dataset

- Run `psql -f week_4/week4_day2_lab.sql` to rebuild the `week4_day2` schema.
- Only lab tables are dropped; anything in `public` stays untouched.
- Running it before class guarantees consistent row counts and plans.

Schema At a Glance

- `students` – roster of 60 learners with majors and entry terms.
- `courses` + `course_offerings` – what runs each term and who teaches it.
- `enrollments` – status, grade points, credits attempted/earned.
- `term_credit_summary` + `enrollment_audit` – start empty for automation demos.

Why the Constraints Matter

- Credit goals stay reasonable thanks to a CHECK on `students`.
- `enforce_completed_grade` blocks a completed row without a grade.
- Foreign keys guarantee enrollments always point to real students/offers.
- Trustworthy constraints make the analytics believable.

Synthetic Data Highlights

- `generate_series` + name arrays produce varied full names.
- Four consecutive terms give each student a transcript story.
- Modular arithmetic sprinkles in completed, enrolled, and dropped rows.
- Completed rows include grade points so GPA math works right away.

Baseline Query: Feel the Cost

- EXPLAIN (ANALYZE, BUFFERS)
  SELECT *
  FROM week4_day2.enrollments
  WHERE offering_id = 18 AND status = 'completed';
- Expect a sequential scan, high buffer hits, and slower timing.
- Screenshot this plan—it becomes the “before” artifact for the lab.

Create the Covering Index

- CREATE INDEX CONCURRENTLY IF NOT EXISTS week4_day2_enrollments_offering_status_idx
  ON week4_day2.enrollments (offering_id, status);
- Follow with `ANALYZE week4_day2.enrollments;` to refresh statistics.
- Explain why `CONCURRENTLY` matters even in a demo (no write lock).

Rerun the Plan

- The same query should now show an Index or Bitmap Heap Scan.
- Point out the reduced timing and buffer counts.
- SELECT relname, idx_scan
  FROM pg_stat_user_indexes
  WHERE schemaname = 'week4_day2'
    AND relname = 'week4_day2_enrollments_offering_status_idx';
- Use the catalog query to prove PostgreSQL actually used the index.

Keep the Planner Informed

- Lab environments may need manual `ANALYZE`.
- SELECT attname, n_distinct
  FROM pg_stats
  WHERE schemaname = 'week4_day2' AND tablename = 'enrollments';
- Share on the midterm proposal which indexes and stats you rely on.

Window Functions: Big Idea

- Start with a normal GROUP BY, then layer “bonus columns.”
- `PARTITION BY` resets the calculation for each group you choose.
- `ORDER BY` inside the window controls the sequence of rows.
- Our bonus columns: ranking plus credits earned so far.

Pass 1 – Plain Aggregate

- SELECT s.full_name,
  o.term,
  SUM(e.grade_points * e.credits_attempted) / NULLIF(SUM(e.credits_attempted), 0) AS term_gpa,
  SUM(e.credits_earned) AS term_credits
  FROM week4_day2.enrollments e
  JOIN week4_day2.course_offerings o ON o.offering_id = e.offering_id
  JOIN week4_day2.students s ON s.student_id = e.student_id
  WHERE e.status = 'completed'
  GROUP BY s.full_name, o.term
  ORDER BY o.term, term_gpa DESC;
- Describe this as the term scoreboard for each student.

Pass 2 – Add a Rank

- WITH per_term AS (
  -- use Pass 1 query here
)
  SELECT *,
  ROW_NUMBER() OVER (PARTITION BY term ORDER BY term_gpa DESC) AS term_rank
  FROM per_term
  ORDER BY term, term_rank;
- Message: PARTITION BY term simply restarts the numbering every term.
- Encourage students to read the result aloud to cement the idea.

Pass 3 – Running Credits

- WITH per_term AS (
  -- same Pass 1 query
)
  SELECT full_name,
  term,
  term_gpa,
  term_rank,
  SUM(term_credits)
  OVER (PARTITION BY full_name ORDER BY term
  ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS credits_so_far
  FROM per_term
  ORDER BY term, term_rank;
- Explain the window in plain English: “For this student, add earlier rows.”
- Show a 3–4 row sample so everyone can visualise the output.

Sample Output

- full_name | term | term_gpa | term_rank | credits_so_far
- Andrea Flores | 2023-Fall | 3.67 | 1 | 12
- Brian Kim | 2023-Fall | 3.20 | 2 | 9
- Use a screenshot or reading exercise so students connect the dots.

Save It as a View

- CREATE OR REPLACE VIEW week4_day2.vw_student_term_progress AS <pass 3 query>;
- Views are just saved SELECT statements—nothing mysterious.
- Mention optional comments and simple grants for future you.

Share the View

- GRANT SELECT ON week4_day2.vw_student_term_progress
  TO reporting_role;
- Hand analysts the view, not the raw tables.
- If needed later, add `security_barrier = true` for stricter filtering.

Trigger Blueprint

- Goal: whenever a grade changes, refresh the summary table.
- Attach an `AFTER INSERT OR UPDATE` trigger to enrollments.
- If the trigger fails, the original UPDATE is rolled back automatically.

Simpler update_term_summary()

- ```
  CREATE OR REPLACE FUNCTION week4_day2.update_term_summary()
  RETURNS trigger
  LANGUAGE plpgsql AS $$
  DECLARE
      v_term text;
      v_attempted int;
      v_earned int;
      v_quality numeric(10,2);
  BEGIN
      SELECT term INTO v_term
      FROM week4_day2.course_offerings
      WHERE offering_id = NEW.offering_id;

      SELECT COALESCE(SUM(e.credits_attempted), 0),
             COALESCE(SUM(e.credits_earned), 0),
             COALESCE(SUM(COALESCE(e.grade_points, 0) * e.credits_attempted), 0)
      INTO v_attempted, v_earned, v_quality
      FROM week4_day2.enrollments e
      JOIN week4_day2.course_offerings o ON o.offering_id = e.offering_id
      WHERE e.student_id = NEW.student_id
        AND o.term = v_term
  ```
  ```

Attach the Trigger & Test

- CREATE TRIGGER enrollment_finalize_summary
  AFTER INSERT OR UPDATE OF status, grade_points, credits_earned
  ON week4_day2.enrollments
  FOR EACH ROW EXECUTE FUNCTION week4_day2.update_term_summary();
- Demo: mark enrollment 25 as completed, then query the summary table.
- Optional stretch: insert into `enrollment_audit` for change history.

Advisor Helper Function

- ```
  CREATE OR REPLACE FUNCTION week4_day2.fn_flag_at_risk(
      p_term text,
      p_min_gpa numeric,
      p_min_credits int)
  RETURNS TABLE (
      student_id uuid,
      full_name text,
      term text,
      term_gpa numeric,
      credits_so_far int)
  LANGUAGE sql AS $$
      SELECT v.student_id,
             v.full_name,
             v.term,
             v.term_gpa,
             t.credits_earned AS credits_so_far
      FROM week4_day2.vw_student_term_progress v
      JOIN week4_day2.term_credit_summary t
        ON t.student_id = v.student_id AND t.term = v.term
      WHERE v.term = p_term
        AND (v.term_gpa < p_min_gpa OR t.credits_earned < p_min_credits)
  ```
  ```

Lab Deliverables

- Before/after `EXPLAIN` screenshots for the indexing step.
- Query output from the window-function view.
- Rows from `term_credit_summary` after running the trigger test.
- Results from the advisor helper function with your thresholds.
- Reflection prompt: “Indexing mattered most when ____.”

Wrap-Up & Next Steps

- Double-check trigger math and audit rows before submitting the lab.
- Document the index, view, trigger, and function for the midterm proposal.
- Experiment with new thresholds or extra columns if time remains.
- Bring any open questions to the proposal workshop.